Quest 4: The Rankening

Lord of the Wings · scoring the Tournament of Elven Warriors

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. Every number in the examples and traces was checked by running the code in this repo (`node partN.ts --sample`, plus small instrumented copies to print intermediate values).

In one sentence: turn tournament records into a ranking, then compute a checksum: Σ position × (the value being ranked).
Input: one record per line, such as `id=7 quest=1 part=2 time=345`, in any order.
Output: the checksum.
What changes between parts: what the ranking is based on (total time → parts completed then time → points with ties).

1. The story

The Grand Council agrees to help the journey to Nulldor, where the cursed Dragonduck egg Frodoff is carrying must be destroyed. The elves want to send their best warrior, but nobody has worked out the standings of the **Tournament of Elven Warriors** yet. You build the leaderboard.

2. Understanding the question

The records

Field	Meaning
`id`	the participant
`quest`	which tournament quest (one quest per day)
`part`	part 1, 2 or 3 of that quest
`time`	seconds taken for that part

Records are **in no particular order**, so you have to group them yourself, by participant and later by quest.

The checksum

```
checksum = 1 × (value of 1st place) + 2 × (value of 2nd place) + 3 × (value of 3rd place)
+ …
```

The "value" is total time in Parts I and II, and total points in Part III.

Reading the records (same in all parts)

```
10  const inputObjs: { [key: string]: number }[] = INPUTS.map((i) =>
11    Object.fromEntries(
12      i.split(" ").map((s) => {
13        const [key, value] = s.split("=");
14        return [key, Number(value)];
15      }),
16    ),
17  );
```

q4/part1.ts, lines 10–17

Lines	What happens
12	Splits a record on spaces: `["id=1", "quest=1", "part=1", "time=187"]`.
13–14	Splits each piece on `=` into `[key, Number(value)]`.
11	`Object.fromEntries` builds `{{id: 1, quest: 1, part: 1, time: 187}}`.

3. Part I: one quest, rank by total time

The question

Every participant finished all three parts of one quest. Add each participant's three times and rank from **shortest total to longest**. There are no ties.

Worked example (sample)

Position	id	Times	Total	Position × total
1	1	187 + 299 + 362	848	848
2	5	257 + 346 + 744	1347	2694
3	2	192 + 562 + 745	1499	4497
4	3	413 + 455 + 1168	2036	8144
5	4	457 + 558 + 1268	2283	11415
Checksum				**27598**

Step-by-step solution

1. Parse each record into `{id, quest, part, time}`.
2. For each record, add its time to that participant's total.
3. Sort the participants by total time, smallest first.
4. Go down the list: position 1, 2, 3, …. Add `position × total time` for each.
5. That sum is the checksum.

My solution

```
19  const ranks: { [key: string]: number } = {};
20
21  inputObjs.forEach((i) => {
22    const key = String(i["id"]);
23    ranks[key] = (ranks[key] || 0) + (i["time"] ?? 0);
24  });
25
26  const result = Object.entries(ranks)
27    .sort((a, b) => a[1] - b[1])
28    .map(([, i]) => i)
29    .reduce((a, b, idx) => a + b * (idx + 1), 0);
30
31  console.log(result);
```

q4/part1.ts, lines 19–31

Lines	What happens
19	`ranks` maps a participant id to its running total time.
21–24	For each record, adds its time to that participant's total. `ranks[key] || 0` starts a new participant at 0. Order doesn't matter, because addition doesn't care about order.
26–27	`Object.entries` gives `[id, total]` pairs, sorted by total, smallest first.
28	Keeps only the totals: `[848, 1347, 1499, 2036, 2283]`.
29	`reduce` adds `total × (idx + 1)`. The array index is 0-based, so position is `idx + 1`.

Trace: the `ranks` object after line 24

```
{ "1": 848, "2": 1499, "3": 2036, "4": 2283, "5": 1347 }
```

4. Part II: whole tournament, more parts completed comes first

The question

Now the records cover all 20 quests, and most participants didn't finish everything. The ranking rule is:

1. **More completed parts always ranks higher**, whatever the time.
2. With the same number of parts, the **shorter total time** ranks higher.

The checksum still uses total time. There are no ties.

Worked example (sample)

Position	id	Parts	Total time	Position × time
1	1	5	980	980
2	2	5	990	1980
3	3	4	590	1770
4	4	3	435	1740
Checksum				**6470**

id=4 has the **lowest** time (435) but ranks **last**, because it completed only 3 parts. id=1 beats id=2 on time, since both completed 5 parts.

Step-by-step solution

1. Parse each record.
2. For each participant, keep two running numbers: total time, and how many records (parts) they have.
3. Sort by parts, most first. If two participants have the same number of parts, put the smaller total time first.
4. Add `position × total time` down the list.

My solution

```
19  const ranks: { [key: string]: { time: number; parts: number } } = {};
20
21  inputObjs.forEach((i) => {
22    const key = String(i["id"]);
23    const time = i["time"] ?? 0;
24
25    const prev = ranks[key] ?? { time: 0, parts: 0 };
26    ranks[key] = {
27      time: prev.time + time,
28      parts: prev.parts + 1,
29    };
30  });
31
32  const result = Object.entries(ranks)
33    .sort((a, b) => b[1]["parts"] - a[1]["parts"] || a[1]["time"] - b[1]["time"])
34    .map(([, i]) => i)
35    .reduce((a, b, idx) => a + b.time * (idx + 1), 0);
36
37  console.log(result);
```

q4/part2.ts, lines 19–37 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
19	Now each participant stores two numbers: `{{ time, parts }}`.
25–29	For each record: adds the time and adds 1 to the parts count. Each record is one completed part.
33	The two-level sort. `b.parts - a.parts` puts more parts first. If that is `0` (same number of parts), `||` falls back to `a.time - b.time`, which puts shorter time first. This works because `0` is falsy in JavaScript.
35	The checksum uses `b.time`, the total time, times the position.

Trace: `ranks` after line 30

```
{ "1": { time: 980, parts: 5 }, "2": { time: 990, parts: 5 },
  "3": { time: 590, parts: 4 }, "4": { time: 435, parts: 3 } }
```

5. Part III: ranking points, with ties

The question

The final ranking is now based on **points**. For every quest, a separate mini-ranking is made after **each part**:

1. **Who is included:** everyone who completed that part. Anyone who didn't gets 0 points for it.
2. **What is compared:** the **combined** time so far. After part 2 that is part 1 + part 2; after part 3 it is all three.
3. **Points:** first place gets **50** (part 1), **100** (part 2) or **150** (part 3). Each lower position gets 1 point less, down to 1, and anyone below that gets 0. So `points = max(50 × part − (position − 1), 0)`.
4. **Ties:** equal combined times **share a position** and each tied participant gets that position's points. The next participant's position **skips ahead**: 1, 1, 3, not 1, 1, 2.

Add up each participant's points, rank by total points (highest first), and compute `Σ position × points`. **Tied totals share a position here as well:** `… + 55×374 + 55×374 + 57×…`.

Worked example with ties (sample3.txt, which your code uses)

Quest · part	Combined times	Positions → points
Q1 · 1	id2 90, id3 90, id4 110, id1 120	1st id2 50 · **1st** id3 50 · **3rd** id4 48 · 4th id1 47
Q1 · 2	id2 260, id3 260, id4 280, id1 300	1st id2 100 · 1st id3 100 · 3rd id4 98 · 4th id1 97
Q1 · 3	nobody finished	-
Q2 · 1	id3 120, id1 130, id2 130, id4 150	1st id3 50 · 2nd id1 49 · 2nd id2 49 · 4th id4 47
Q2 · 2	id1 340, id3 340, id2 350	1st id1 100 · 1st id3 100 · 3rd id2 98
Q2 · 3	id1 670, id2 700	1st id1 150 · 2nd id2 149

Position	id	Points	Total	Position × points
1	2	50 + 100 + 49 + 98 + 149	446	446
2	1	47 + 97 + 49 + 100 + 150	443	886
3	3	50 + 100 + 50 + 100	300	900
4	4	48 + 98 + 47	193	772
Checksum				**3004**

The puzzle's other example (the Part II data, with no ties) gives 441 / 444 / 300 / 193 and a checksum of **2998**. Running your Part III code on `sample2.txt` gives 2998 as well.

Step-by-step solution

1. Parse each record and file its time under `[quest][participant][part - 1]`.
2. For each quest, and each part k = 1, 2, 3: collect everyone who completed part k, with their **combined** time for parts 1…k.
3. Sort that list, fastest first. Give out positions, where equal times share a position and the next one skips ahead (1, 1, 3).
4. Each participant gets `max(50·k - (position - 1), 0)` points, added to their total score.
5. When every quest and part is done, sort the participants by total score, highest first. Give positions with the same tie rule.
6. Add `position × score` down the list. That is the checksum.

My solution, step 1: group times by quest and participant

```
19  const ranks: { [quest: string]: { [id: string]: number[] } } = {};
20
21  inputObjs.forEach((i) => {
22    const id = i["id"] ?? 0;
23    const quest = i["quest"] ?? 0;
24    const time = i["time"] ?? 0;
25    const part = i["part"] ?? 0;
26
27    const prevQuest = ranks[quest] ?? {};
28    const prevId = prevQuest[id] ?? [];
29    prevId[part - 1] = time;
30
31    ranks[quest] = { ...prevQuest, [id]: prevId };
32  });
```

q4/part3.ts, lines 19–32

Lines	What happens
19	`ranks[quest][id]` is an array of that participant's times for parts 1–3 of that quest.
27–29	Gets (or creates) the quest's object and the participant's array, then puts the time at index `part - 1`. Because of this, the order of the records doesn't matter.
31	Writes the updated object back. The spread `{{...prevQuest}}` keeps the other participants.

Trace: `ranks` after line 32 (sample3)

```
{ "1": { "1": [120,180], "2": [90,170], "3": [90,170], "4": [110,170] },
  "2": { "1": [130,210,330], "2": [130,220,350], "3": [120,220], "4": [150] } }
```

My solution, step 2: points for every (quest, part)

```ts
const scores: Record<string, number> = {};

Object.values(ranks).forEach((r) => {
  for (let i = 0; i < 3; i++) {
    const list = Object.entries(r)
      .filter(([, times]) => times[i] !== undefined)
      .map(([id, times]) => [
        id,
        times.slice(0, i + 1).reduce((a, b) => a + b, 0),
      ]) as [string, number][];

    if (list.length === 0) continue;

    list.sort((a, b) => a[1] - b[1]);

    let rank = 0;

    list.forEach(([id, time], j) => {
      if (j === 0 || time !== list[j - 1]?.[1]) rank = j + 1;

      const points = Math.max(50 * (i + 1) - (rank - 1), 0);
      scores[id] = (scores[id] ?? 0) + points;
    });
  }
});
```

q4/part3.ts, lines 34–58

Lines	What happens
36–37	For each quest `r`, and for each part index `i` = 0, 1, 2 (parts 1, 2, 3)…
39	…keeps only participants who completed this part (`times[i]` exists)…
40–43	…and pairs each one with their **combined** time: `times.slice(0, i + 1)` takes parts 1…i+1, and `reduce` adds them. For quest 2, part 2: id1 → 130 + 210 = 340.
45	Skips a part that nobody completed, such as Q1 · 3.
47	Sorts fastest first.
49–52	**Competition ranking.** `rank` only moves to `j + 1` when the time is different from the previous one. Equal times keep the earlier rank, and the next different time jumps to its real position.
54	`50 × (i + 1)` is the top prize (50, 100, 150); subtract 1 per position below first, and never go below 0.
55	Adds the points to the participant's total score.

Trace: lines 49–55 for Q1 · part 1

j	id	time	same as previous?	rank	points
0	2	90	(first)	1	50
1	3	90	yes → keep	1	50
2	4	110	no → j + 1	3	48
3	1	120	no → j + 1	4	47

My solution, step 3: final ranking and checksum

```
60  let rank = 0;
61
62  const result = Object.entries(scores)
63    .sort((a, b) => b[1] - a[1])
64    .map(([, i]) => i)
65    .reduce((a, b, idx, arr) => {
66      if (idx === 0 || b !== arr[idx - 1]) rank = idx + 1;
67      return a + b * rank;
68    }, 0);
69
70  console.log(result);
```

q4/part3.ts, lines 60–70

Lines	What happens
60	`rank` lives outside the `reduce` so it keeps its value between items.
63–64	Sorts the total scores, highest first, and keeps only the numbers. For the sample: `[446, 443, 300, 193]`.
66	The same tie rule as before: the rank only moves to `idx + 1` when the score changes. So two scores of 374 at index 54 and 55 both get rank 55, and the next score gets 57.
67	Adds `score × rank`.

Easy-to-miss details in Part III, all handled by the code:

- Mini-rankings are made separately for each **(quest, part)**, not across the whole tournament (line 36–37).
- Rankings after part 2 and part 3 use **running totals**, not the time for that part alone (line 42).
- Ties use **1, 1, 3** ranking, both when giving out points (line 52) and in the final checksum (line 66).
- Positions far down still get 0 points rather than negative points (`Math.max(…, 0)`, line 54).

6. Summary

Part	Ranked by	Ties?	Checksum multiplies
I	total time (low first)	no	total time
II	parts completed (high first), then total time (low first)	no	total time
III	total points from per-(quest, part) mini-rankings	yes, 1-1-3 style	total points